



Computer Science Department
Web Application and Technologies (COMP 334)
Second Semester 2025/2026

Assignment 2: Dynamic PHP Web Application

Due Date 04/06/2026

1. Overview

In this assignment you will extend the static Palestinian souvenir store website built in Assignment 1 into a fully **dynamic, database-driven web application** using PHP and MySQL. The application allows customers to browse products, manage a shopping basket, register, log in, and place orders. Employees can manage product inventory.

Important: This assignment focuses on PHP server-side scripting and SQL only. **No CSS stylesheets and no JavaScript are permitted anywhere in your submission.** All visual presentation is produced by the browser's default rendering. All database interaction must use PDO with named prepared statements exclusively.

⚠ **Academic Integrity:** All submitted PHP and SQL code must be your own original work. Code substantially copied from classmates, online sources, AI-generated tools, or any source outside the course materials will result in a mark of zero for the entire assignment. The instructor reserves the right to conduct an oral assessment on any submission to verify authorship.

⚠ **Course Materials Only:** You must use only PHP functions, SQL syntax, and programming constructs covered in the course lectures, slides, textbook, and provided examples. The marker will check your code against course materials. Using functions, libraries, or techniques not taught in the course will result in mark deductions even if the code works correctly.

2. Learning Objectives

By completing this assignment, students will be able to:

- Create data-driven PHP pages that interact with a MySQL database using PDO.
- Use PHP superglobal arrays (\$_GET, \$_POST, \$_SESSION, \$_FILES) correctly and safely.
- Write SQL queries (SELECT, INSERT, UPDATE, DELETE) using only PDO prepared statements with named binding parameters.
- Implement user authentication and session management in PHP.
- Design and apply a PHP class (Product) with appropriate constructors, getters, setters, and display methods.
- Handle file uploads securely in PHP.
- Build paginated and sortable data tables using PHP and SQL.

- Preserve application state across requests using PHP sessions.
- Generate server-side validated HTML forms without JavaScript.

3. Technical Constraints and Requirements

The following rules apply to every file in your submission:

- **No CSS:** Do not use any CSS stylesheet, inline style attributes, or <style> tags.
- **No JavaScript:** Do not include any <script> tags, event attributes (onclick, onsubmit, etc.), or external JS files.
- **PDO only:** All database queries must use PDO with named prepared statements (e.g. :name, :price). Raw mysql_* functions, MySQLi, or string-concatenated SQL are not permitted.
- **Named parameters only:** Use named binding parameters (:param) not positional (?) in all prepared statements.
- **Separate config file:** Database credentials and the PDO object must be defined exclusively in dbconfig.inc.php. All other scripts include this file.
- **Course materials only:** Use only PHP functions and SQL constructs taught in lectures, the textbook, and provided examples.
- **Semantic HTML5:** Use semantic elements (<header>, <nav>, <main>, <article>, <section>, <footer>, <figure>, <address>) appropriately on every page.
- **Relative URLs:** All internal links and resource references must use relative URLs.

4. Database Design

You must create a MySQL database named **webSTUDENTID_souvenirStore** (replace STUDENTID with your actual student number) and populate it with at least **10 product records**, **2 customer records**, and at least **1 employee record** and display their usernames and passwords on the assignment 2 index page for testing.

basket (session-based — no database table required)

The shopping basket is stored in the PHP **\$_SESSION** superglobal array as an associative array keyed by product_id. Each entry holds the product_id, product_name, unit_price, quantity, and the default photo filename. The basket is cleared when the order is placed or the session expires.

5. The Product Class

You must define a **Product** class in a file called **Product.class.php**. This class encapsulates a single product record and provides all HTML rendering methods for product display. All other scripts that need to display a product must use this class.

5.1 Properties

The class must declare the following private properties:

- productId
- productName
- category
- description
- price
- quantity
- rating
- photo1
- photo2
- photo3
- defaultPhoto

5.2 Constructor and Accessors

The class must provide a constructor that accepts all properties as parameters, and a complete set of **getter** and **setter** methods for each property (e.g. *getProductId()*, *setPrice()*).

5.3 Required Methods

displayInTable()

Returns an HTML **<tr>** string containing one table row with the following cells:

- The default product photo displayed as an **** element, sourced from the `images/` folder.
- The `product_id`, hyperlinked to `view.php?id={product_id}` as a GET request.
- The `product_name`.
- The category.
- The price.
- The quantity.
- An Actions cell — its content depends on the logged-in user's role (see Section 8).

displayProductPage()

Returns a **<main>** HTML string that renders the full product detail page. The layout and content requirements are described in Section 8.3.

6. Required Files and Folder Structure

All assignment files must be placed in the **assTwo/** sub-folder on CS Host, under your **ass/** directory.

7. Shared Layout

Every page must include the shared header and footer using **require_once**. These files must not contain any logic; they output HTML only.

7.1 header.inc.php

- A <header> element containing the store name as an <h1> heading and the store logo as an element.
- A <nav> element with links to: products.php, register.php, login.php (shown only when no user is logged in), logout.php (shown only when a user is logged in), and basket.php.
- If a user is logged in, display their name and role (Customer or Employee) in the header.

7.2 footer.inc.php

- A <footer> element that always appears at the bottom of each page.
- The store's postal address, marked up with the <address> element.
- Customer support information: telephone number and email address.
- A link to a Contact Us page.
- The footer must always be visible at the bottom of each page regardless of content length.

8. Script Requirements

8.1 register.php — User Registration

This script is **self-referencing**. When called with no POST data it displays the registration form. When POST data is received it validates the input and, on success, inserts a new record into the *users* table.

Registration Form Fields

Group the fields using <fieldset> and <legend>. The form must collect:

Field	Input Type	Notes
First Name	text input	required
Last Name	text input	required
Email Address	email input	required, must be unique in the database
Mobile Number	text input	required

Date of Birth	date input	required — see validation rules below
Flat / Unit No	text input	optional
Street Name & No	text input	required
City	text input	required
Country	text input	required
Postal Code	text input	required — exactly 6 digits, numbers only
Password	password input	required
Confirm Password	password input	required — must match Password
Role	radio buttons	Customer (default) or Employee

Server-Side Validation Rules

All validation must be performed on the server in PHP. The form must be redisplayed with appropriate error messages if any rule is violated:

- All required fields must be non-empty after trimming whitespace.
- Email must be in a valid format.
- Email must not already exist in the users table.
- Date of birth must be a valid date in the past (before today).
- The customer must be at least 18 years old on the date of registration. Calculate the difference between the date of birth and today's date on the server using PHP date functions covered in the course.
- Postal code must contain exactly 6 numeric digits.
- Password and Confirm Password must be identical.
- Password must be stored hashed.

Successful Registration Response

On successful insertion, the script must display an HTML page that shows the newly generated **10-digit User ID** and a hyperlink to **login.php**. The 10-digit ID must be generated in PHP.

8.2 login.php — Authentication

This script is **self-referencing**. It displays a login form (email + password). On POST, it retrieves the user record by email, verifies the password, and on success starts a PHP session storing at minimum: `user_id`, `first_name`, `last_name`, `role`, and a flag indicating the user is logged in. On failure, redisplay the form with an error message.

Session-preserved return: If the user was redirected to login.php from another script (e.g. from `add_to_basket.php`), the originating URL must be stored in the session before

the redirect. After successful login, the user must be redirected back to that URL automatically.

8.3 products.php — Main Product Listing

This is the main page of the application (*in the assignment 2 index page you must add a link that refers to this script*). It displays a filterable, sortable, paginated table of products. This script must preserve the full search and pagination state in the session as a JSON string.

Page Layout

- A link to add.php labelled "Add Product" (visible to employees only).
- An Advanced Product Search form (see below).
- The Products Table Result (see below).

Advanced Product Search Form

The form must use method POST and action products.php. It must contain the following filter fields:

- Product Name — text input with placeholder "Product Name" (partial match, case-insensitive).
- Maximum Price — number input with placeholder "Max Price" (show products with price <= entered value).
- Category — a <select> drop-down populated dynamically from the database with a default option of "Select Category".
- A Filter submit button.
- A Reset / Show All link that clears the session search state and reloads products.php.

Session-Preserved Search State

When the Filter button is clicked, the search criteria, sort column, sort direction, current page number, and items-per-page selection must be stored in the PHP **\$_SESSION** as a JSON-encoded string. When products.php is loaded without new POST data (e.g. when the user returns from viewing a product), the saved session state must be decoded and used to regenerate the same table the user was viewing before. The session key must be named *product_search_state*.

The JSON object stored in the session must include at minimum:

- name — the entered name filter value (empty string if not set).
- max_price — the entered price filter value (empty string if not set).
- category — the selected category (empty string if not set).
- sort_col — the column currently sorted (product_id, price, or category).
- sort_dir — ASC or DESC.
- current_page — the current page number (integer, starting at 1).
- per_page — the number of records per page (5, 10, 15, 20, or 0 meaning all records).

Sortable Columns

The table headers for **Product ID**, **Price**, and **Category** must be hyperlinks that toggle the sort direction (ASC / DESC) when clicked. Clicking a column that is already sorted ascending must switch it to descending, and vice versa. The active sort column and direction must be preserved in the session state.

Pagination

The search results must be paginated. The page must provide:

- A per-page selector — a `<select>` element with options 5, 10, 15, 20, and All. The currently selected value must be preserved in the session state.
- Navigation links — Previous and Next links, shown only when applicable. Each link must carry all current filter and sort parameters as GET parameters.
- A page indicator showing the current page and total number of pages (e.g. Page 2 of 4).
- The default number of records per page is 5.

Products Table

The table must display the following columns:

- Product Image — the default photo as an `<img>` element.
- Product ID — hyperlinked to `view.php?id={product_id}`.
- Product Name.
- Category.
- Price.
- Quantity.
- Actions — role-dependent (see below).

Role-Based Action Buttons

User Role	Actions Displayed
Guest User (Not logged)	View button only (links to <code>view.php?id=...</code>), Add to Basket button (links to <code>add_to_basket.php?id=...</code>)
Customer	View button and Add to Basket button (links to <code>add_to_basket.php?id=...</code>)
Employee	View button, Edit button (links to <code>edit.php?id=...</code>). No Add to Basket.

Product details must be fetched from the database into **Product objects** using PDO's `fetchObject()` method with the Product class name as the argument. Each row in the table must be generated by calling the product's `displayInTable()` method.

8.4 view.php — Product Detail Page

This script receives the product ID via GET (?id=...). It retrieves the product from the database, creates a Product object, and calls *displayProductPage()* to render the page. If the ID is missing or does not exist in the database, display a user-friendly HTML error message.

Product Detail Form Layout

The product details must be displayed inside an HTML form. All fields must be **disabled** (read-only) **except** the editable fields, which are only enabled for employees. The form must include:

Field	Element	Notes
Product ID	text input	Always disabled
Product Name	text input	Always disabled
Category	select	Always disabled
Description	textarea	Disabled for customers; enabled for employees
Price	number input	Disabled for customers; enabled for employees
Quantity	number input	Disabled for customers; enabled for employees
Rating	text input	Always disabled
Photos	radio buttons	Three radio buttons labelled Photo 1, Photo 2, Photo 3. The currently default photo's radio button is pre-selected. Enabled for employees only.

For employees, the form must include a **Save Changes** submit button (if the employee chose the edit action). The form must POST to **edit.php** with the product ID included as a hidden field.

Customers see only a **Back to Products** link and an **Add to Basket** button.

8.5 add.php — Add New Product (Employees Only)

This script is **self-referencing** and accessible to employees only. If a non-employee or unauthenticated user accesses it, redirect them to **login.php**. When called with no POST data it displays the add product form. When POST data is received it validates and inserts the new product.

Add Product Form Fields

- Product Name — text input, required.
- Category — <select> drop-down with predefined categories, required.
- Price — number input, required, must be a positive value.
- Quantity — number input, required, must be a non-negative integer.
- Rating — number input (1–5), required.
- Description — <textarea>, required.
- Photo 1, Photo 2, Photo 3 — three separate file inputs. All three are required. Only files of type image/jpeg are accepted. Each file is saved to the images/ folder and renamed to {product_id}_1.jpeg, {product_id}_2.jpeg, and {product_id}_3.jpeg respectively, where {product_id} is the AUTO_INCREMENT ID returned after insertion.
- Default Photo — radio buttons (Photo 1, Photo 2, Photo 3) to select which image is shown by default in the product table.
- An Insert submit button.

8.6 edit.php — Edit Product (Employees Only)

Accessible to employees only. Receives the product ID via GET (?id=...) and retrieves the record from the database. Displays a form pre-filled with the current values. Only the following fields are editable:

- Price.
- Quantity.
- Description.
- Default Photo — displayed as radio buttons (Photo 1, Photo 2, Photo 3) with the current default pre-selected.
- Replace Photo — three optional file inputs, one per photo slot. If a file is uploaded, it replaces the existing photo for that slot in the images/ folder and in the database. If no file is chosen for a slot, the existing photo is kept unchanged.

Non-editable fields (Product ID, Product Name, Category, Rating) must be displayed as disabled form fields. When the form is submitted (POST to edit.php), validate the input server-side and update the database using a prepared UPDATE statement.

8.7 add_to_basket.php — Add Item to Shopping Basket

This script receives the product ID via GET (?id=...). It must first check whether the user is logged in. If not, it must save the current URL (including the product ID) in the session and redirect to **login.php**. After login, the user is returned to this script automatically.

If the user is logged in (and is a customer), the script displays an Add to Basket confirmation form with:

- Product ID — displayed as a disabled text field (pre-filled).
- Product Name — displayed as a disabled text field (pre-filled).
- Unit Price — displayed as a disabled text field (pre-filled).
- Quantity — a number input, defaulting to 1. The user may change this value.
- An Add to Basket submit button.

On POST, the script must:

- Validate that the requested quantity is a positive integer.
- Retrieve the current available quantity from the database using a prepared SELECT statement.
- Reject the request with an error message if the requested quantity exceeds the available stock.
- Add the item to the session basket array (or increase the quantity if the item already exists in the basket).
- Redirect to basket.php.

8.9 basket.php — Shopping Basket

Displays the current contents of the session basket as an HTML table. The table must include the following columns:

- Product Image (default photo).
- Product Name.
- Unit Price.
- Quantity.
- Line Total (unit price × quantity).
- A Remove link that calls a remove-from-basket action and reloads basket.php.

Below the table, the page must display:

- The basket total — the sum of all line totals, formatted to two decimal places.
- A Continue Shopping link that returns to products.php (the session search state is preserved so the same search result reappears).
- A Proceed to Checkout link that goes to checkout.php.
- If the basket is empty, display an appropriate message and a link back to products.php.

8.10 checkout.php — Checkout and Order Placement

This script must check that the user is logged in and that the basket is not empty; otherwise redirect appropriately. It displays the basket summary table (same format as basket.php) followed by an order placement form.

Credit Card Form Fields

- Cardholder Name — text input, required.
- Card Number — text input, required (16 digits, validated server-side using ctype_digit and strlen).
- Expiry Month — select (01–12), required.
- Expiry Year — select (current year to current year + 10), required.
- CVV — text input, required (3 digits).
- A Place Order submit button.

Order Processing (on POST)

When the form is submitted, the script must:

- Validate all credit card fields server-side.
- Insert a record into the orders table and retrieve the generated order ID.
- Generate a 10-digit order ID.
- Insert one record per basket item into the order_items table.
- Clear the session basket.
- Display a confirmation page showing the 10-digit Order ID and the order total.
- Do not store any actual (لا تخزن معلومات بطاقة ائتمان حقيقية) credit card data in the database.

9. Submission Requirements

9.1 CS Host Deployment

- All files must be uploaded to your CS Host account under the path `public_html/ass/assTwo/`.
- The application must be live and accessible at:
`http://web{STUDENT_ID}.studentprojects.ritaj.ps /ass/assTwo/products.php`
- Update your assignments index page (`ass/index.html`) by adding a link to `assTwo/products.php`.
- All navigation links, images, and form actions must work correctly on the server.

9.2 Database

- Create the database on the server named `webSTUDENTID_souvenirStore`.
- Populate the database with at least 10 product records, 2 customer records, and at least 1 employee record and display their usernames and passwords on the assignment 2 index page for testing.
- Export the full database schema and data as a single SQL file named `students_YOURID.sql` and include it in your ZIP archive.

9.3 ZIP Archive

Compress all assignment files (including the SQL file) into a single archive named **assTwo-stID.zip** (replace stID with your student ID) and submit it to ITC before the due date.

Submission Deadline: Due date is 04/06/2026. Work will be graded only if it is correctly submitted to both CSHost AND ITC. No late submissions will be accepted.